



Classification II: Decision Tree

Lecture 6

肖升生 博士

Dr. Shengsheng Xiao

Email: Xiao.shengsheng@shufe.edu.cn

Classification

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training set



Learn classifier



Model



Predicted classes

Different Classifiers



Refund	Marital Status	Taxable Income	Cheat
No	Single	75K	?
Yes	Married	50K	?
No	Married	150K	?
Yes	Divorced	90K	?
No	Single	40K	?
No	Married	80K	?

Test set



- Decision Tree
 - ID3, C4.5, CART
 - Entropy/Gini Index/Maximum Error
 - Pruning and overfitting
- Regression Tree
- Case Study



Outline

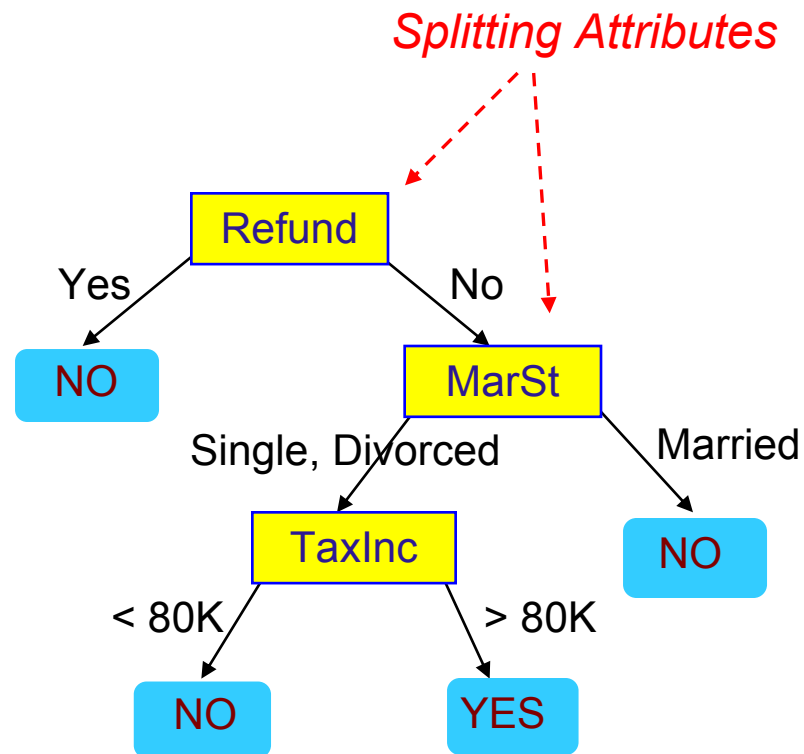
- Decision Tree
 - ID3, C4.5, CART
 - Entropy/Gini Index/Maximum Error
 - Pruning and overfitting
- Regression Tree
- Case Study

Example of a Decision Tree

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data

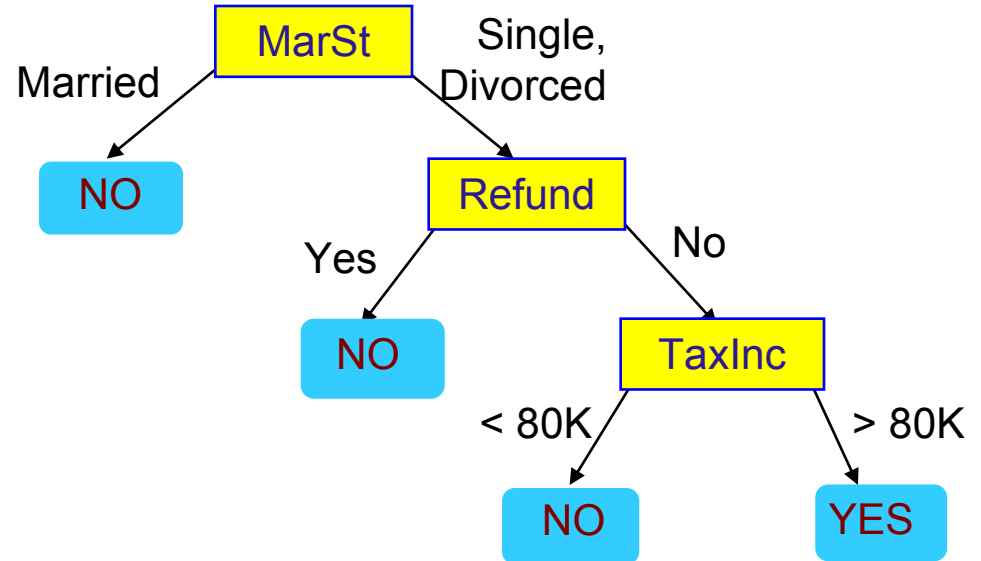


Model: Decision Tree

Example of Decision Tree

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

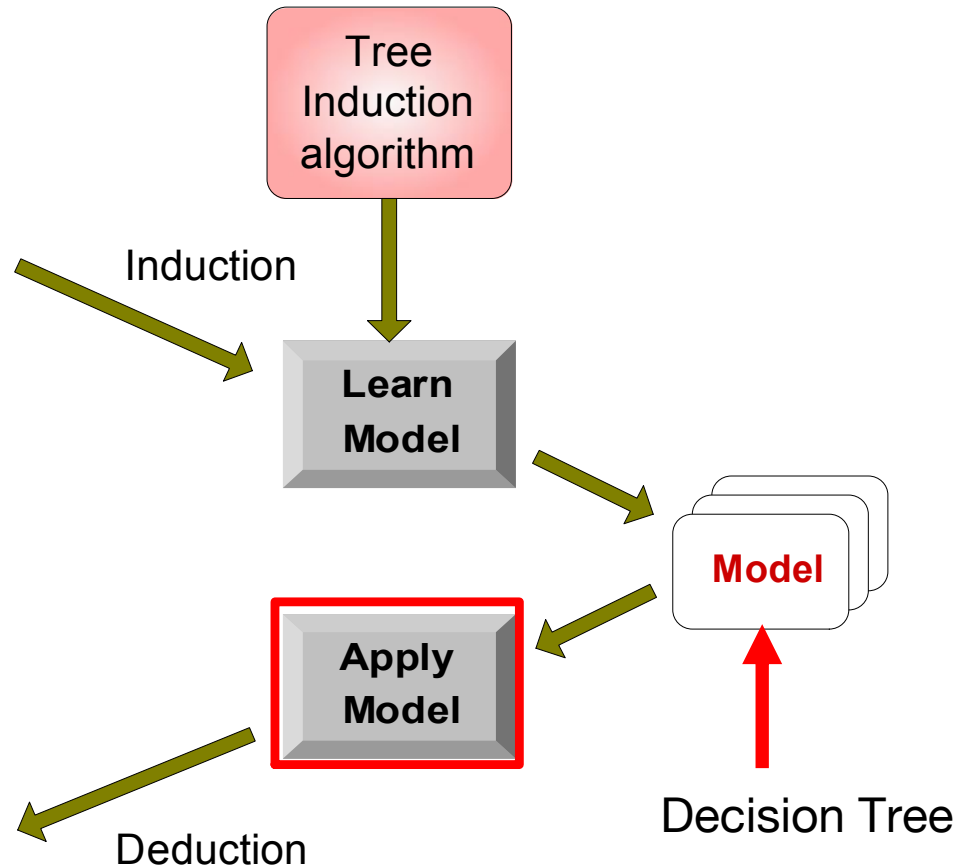
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

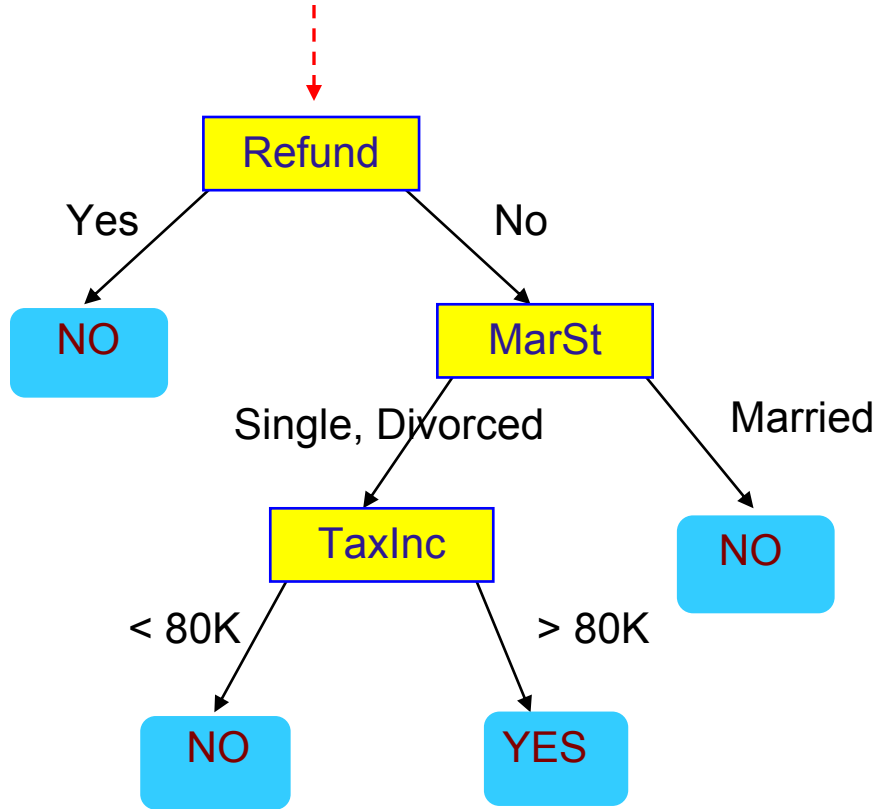
Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Apply Model to Test Data

Start from the root of tree.



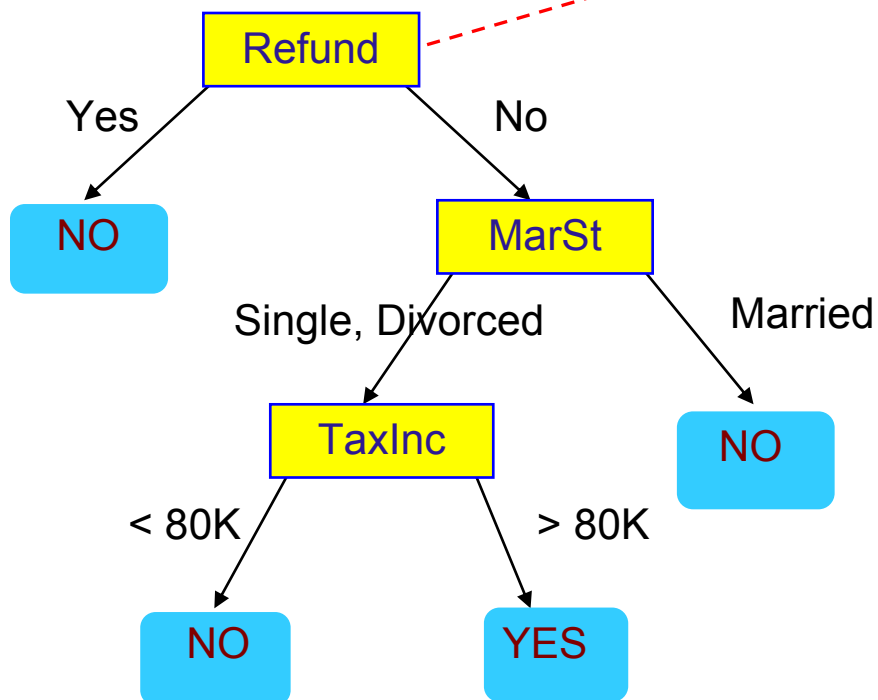
Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Apply Model to Test Data

Test Data

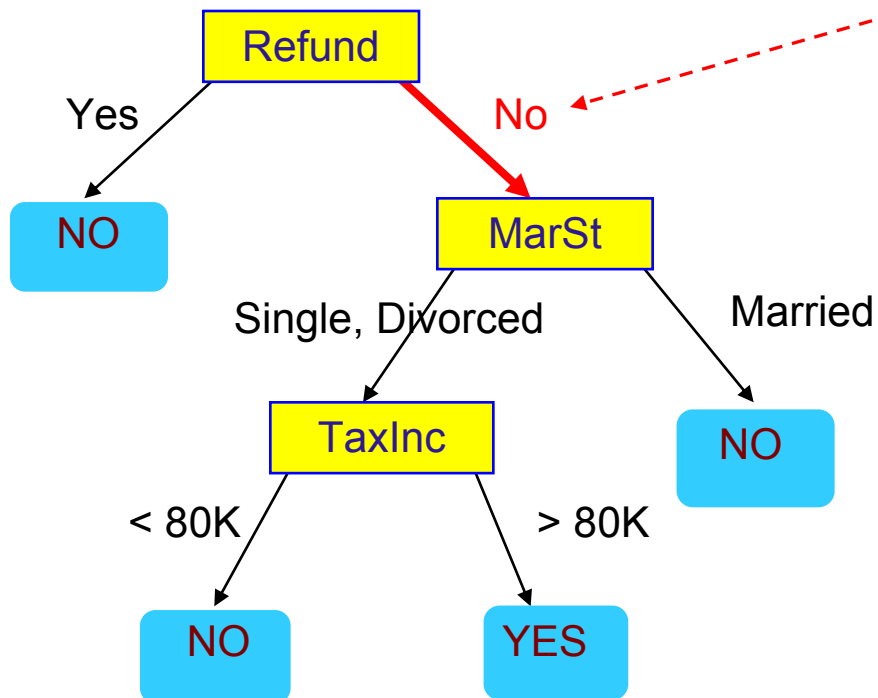
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

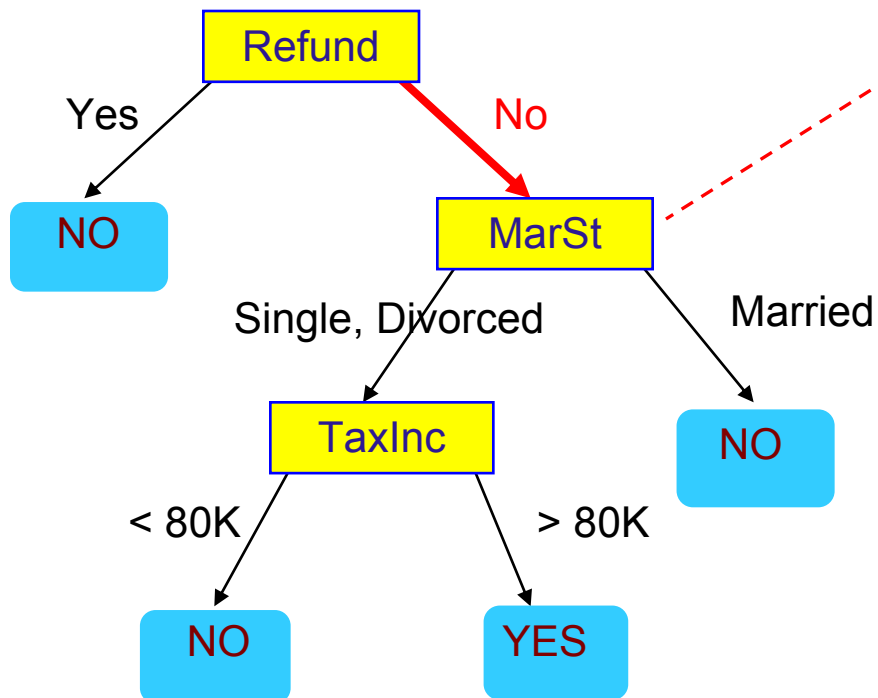
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

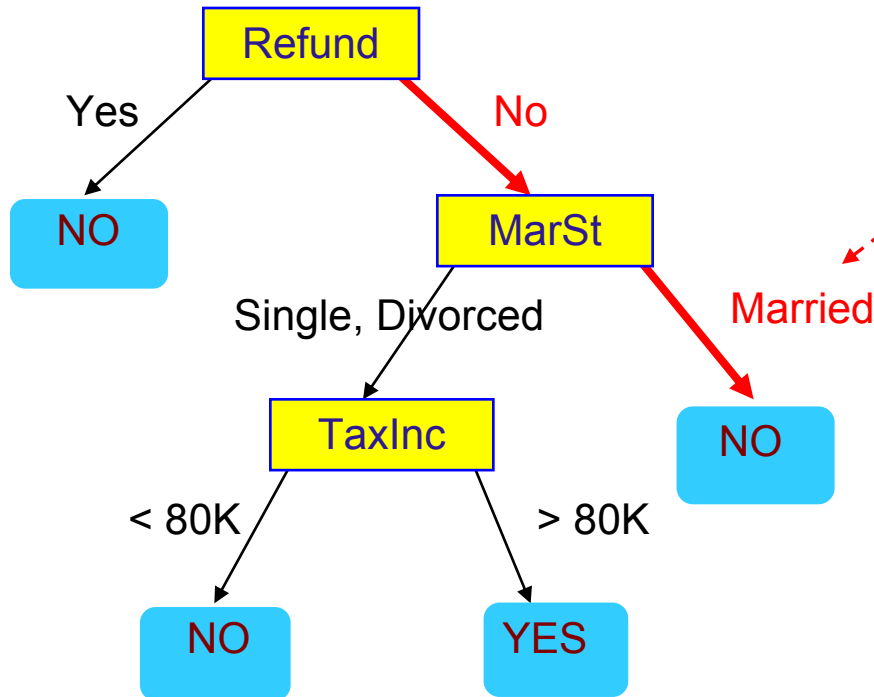
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

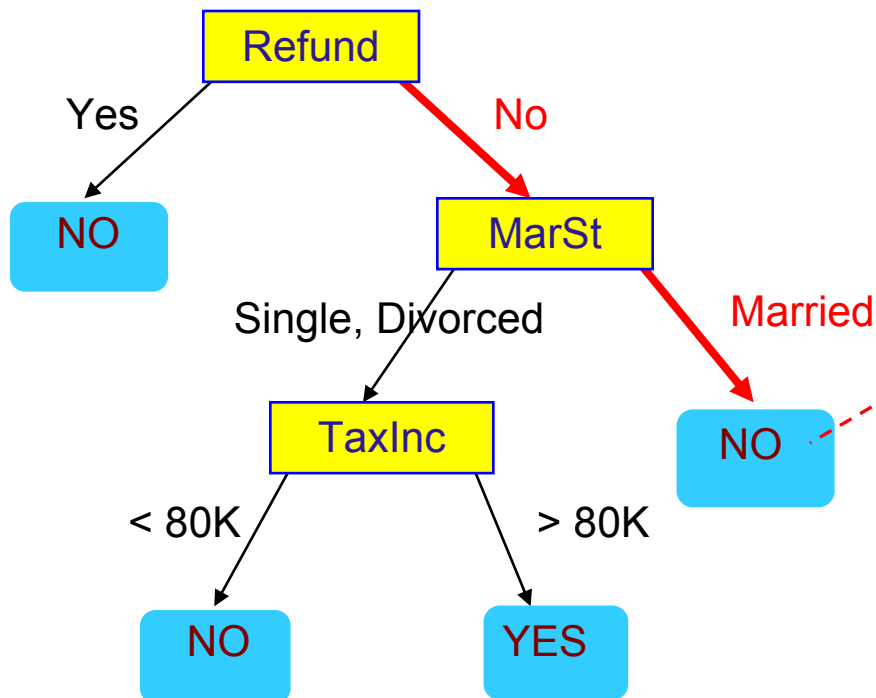
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Assign Cheat to "No"

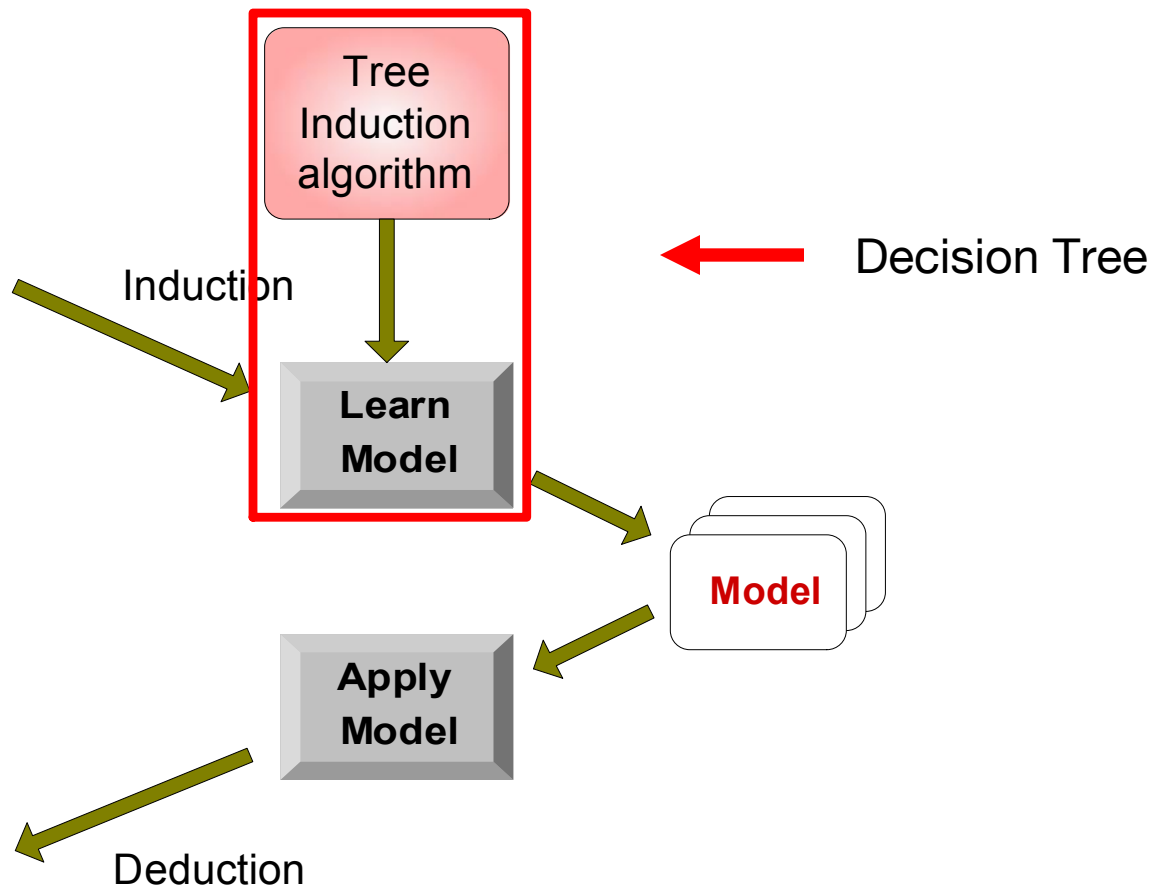
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set





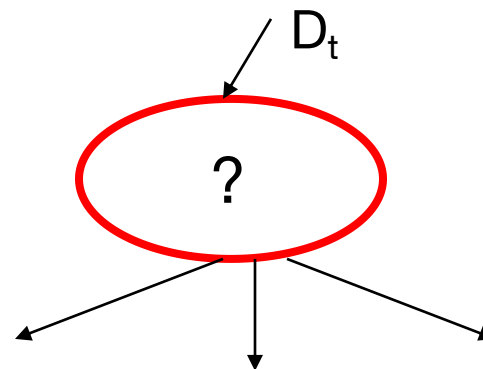
Decision Tree Induction

- Many Algorithms:
 - Hunt's Algorithm
 - ID3
 - C4.5
 - CART
 - ...

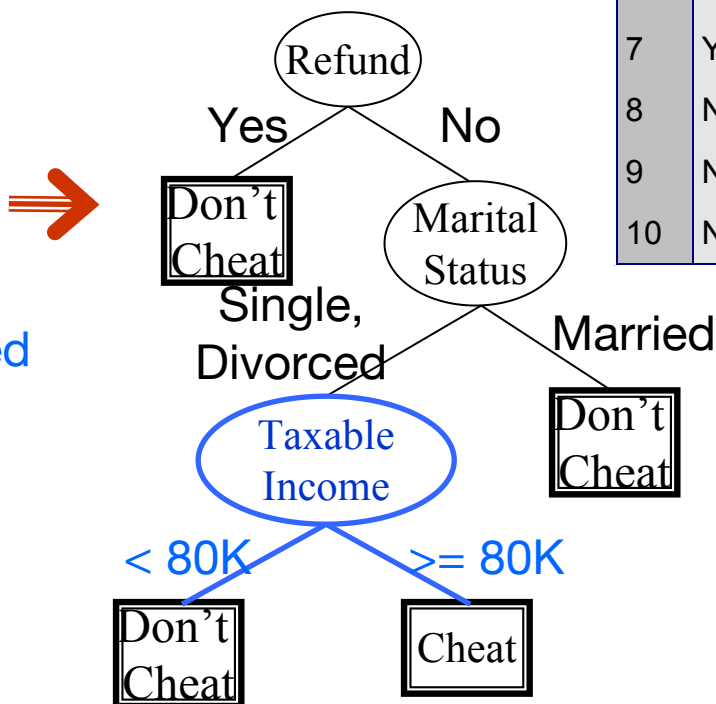
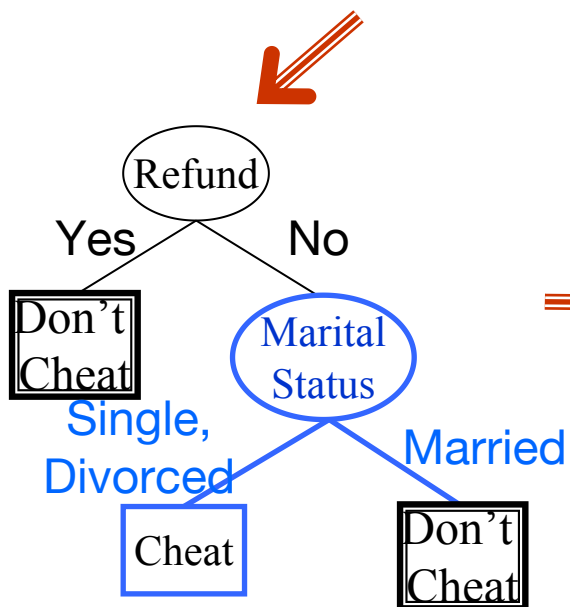
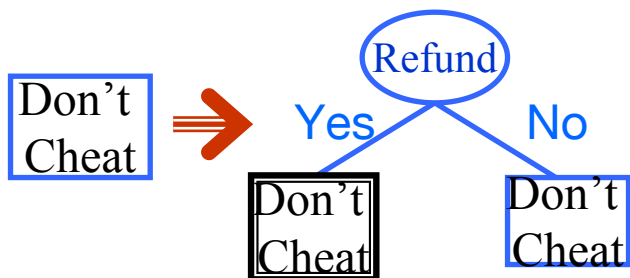
General Structure of Hunt's Algorithm

- Let D_t be the set of training records that reach a node t
- General Procedure:
 - If D_t contains records that belong to the same class y_t , then t is a leaf node labeled as y_t
 - If D_t is an empty set, then t is a leaf node labeled by the default class, y_d
 - If D_t contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset.

<i>Tid</i>	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Hunt's Algorithm



Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Tree Induction

- Greedy strategy.
 - Split the records based on an attribute that optimizes certain criterion.
- Problem to be solved:
 - Determine how to split the records
 - How to specify the attribute test condition?
 - How to determine the best split?
 - Determine when to stop splitting

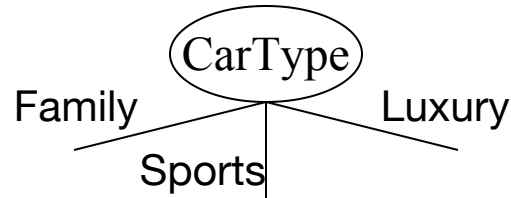


How to Specify Test Condition?

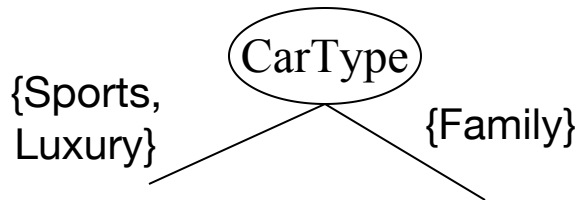
- Depends on attribute types
 - Nominal
 - Ordinal
 - Continuous
- Depends on number of ways to split
 - 2-way split
 - Multi-way split

Splitting Based on Nominal Attributes

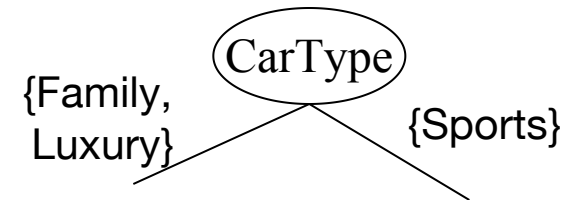
- **Multi-way split:** Use as many partitions as distinct values.



- **Binary split:** Divides values into two subsets. Need to find optimal partitioning.

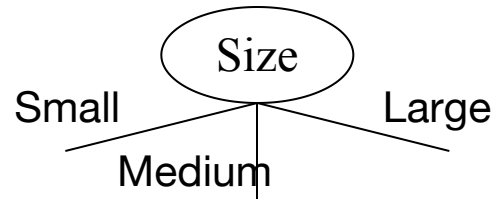


OR

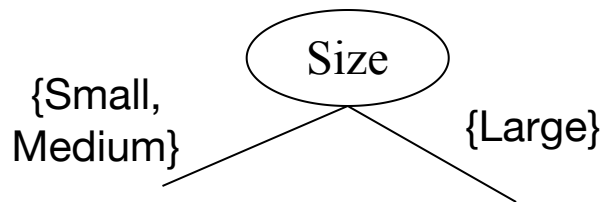


Splitting Based on Ordinal Attributes

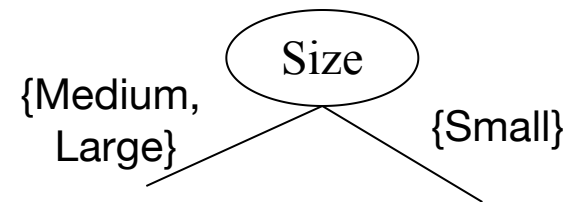
- **Multi-way split:** Use as many partitions as distinct values.



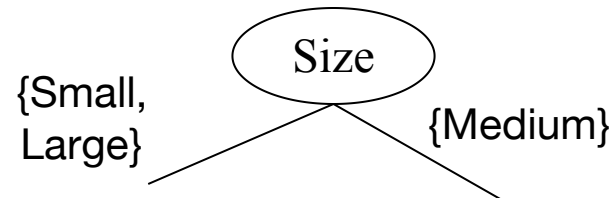
- **Binary split:** Divides values into two subsets. Need to find optimal partitioning.



OR



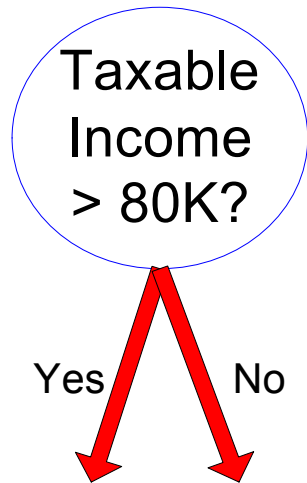
- What about this split?



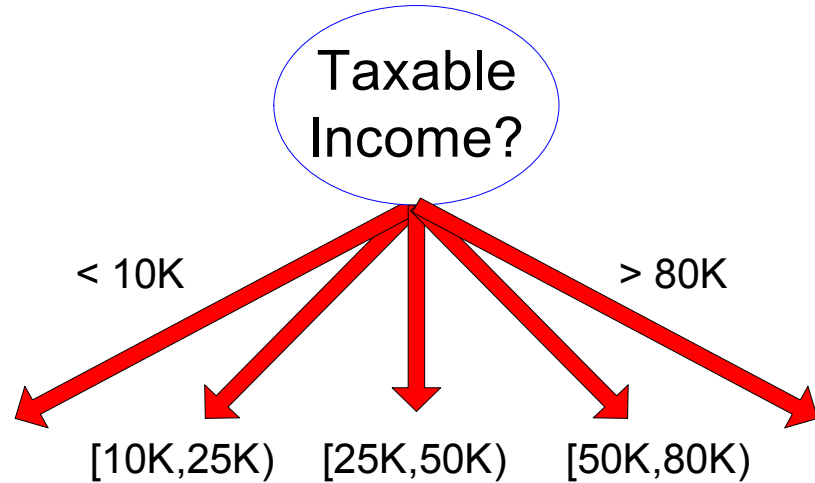
Splitting Based on Continuous Attributes

- Different ways of handling
 - **Discretization** to form an ordinal categorical attribute
 - Static – discretize once at the beginning
 - Dynamic – ranges can be found by equal interval bucketing, equal frequency bucketing (percentiles), or clustering.
 - **Binary Decision**: $(A < v)$ or $(A \geq v)$
 - consider all possible splits and finds the best cut
 - can be more compute intensive

Splitting Based on Continuous Attributes



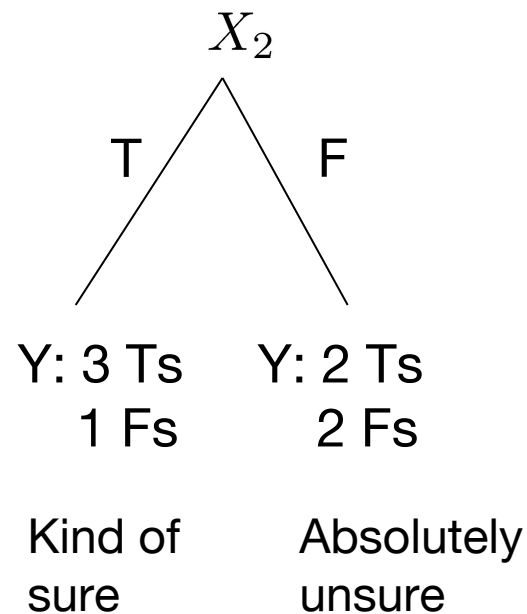
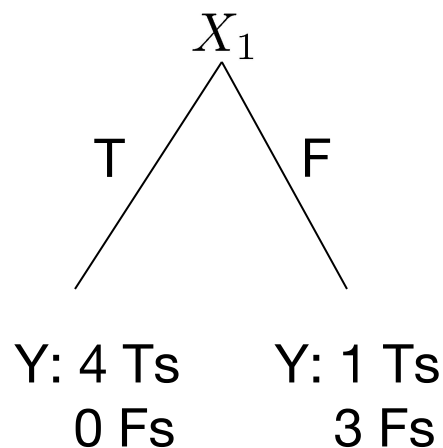
(i) Binary split



(ii) Multi-way split

How to determine the Best Split

X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F



Good split if we are more certain about classification after split –
 Uniform distribution of labels is bad

How to determine the Best Split

- Greedy approach:
 - Nodes with **homogeneous** class distribution are preferred
- Need a measure of node impurity:

C0: 5 C1: 5

Non-homogeneous,
High degree of impurity

C0: 9 C1: 1

Homogeneous,
Low degree of impurity

Measures of Node Impurity

Where $\mathbf{p}_i(\mathbf{t})$ is the frequency of class $\mathbf{i}$ at node $\mathbf{t}$, and $\mathbf{C}$ is the total number of classes

- Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

- Entropy

$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

- Misclassification error

$$Classification\ error = 1 - \max[p_i(t)]$$

Finding the Best Split

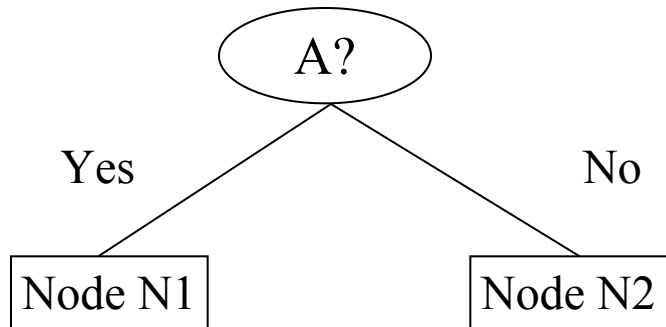
- Compute impurity measure (P) before splitting
- Compute impurity measure (M) after splitting
 - Compute impurity measure of each child node
 - M is the weighted impurity of child nodes
- Choose the attribute test condition that produces the highest gain
 - $\text{Gain} = P - M$
 - or equivalently, lowest impurity measure after splitting (M)

How to Find the Best Split

Before Splitting:

C0	N00
C1	N01

→ M0



C0	N10
C1	N11

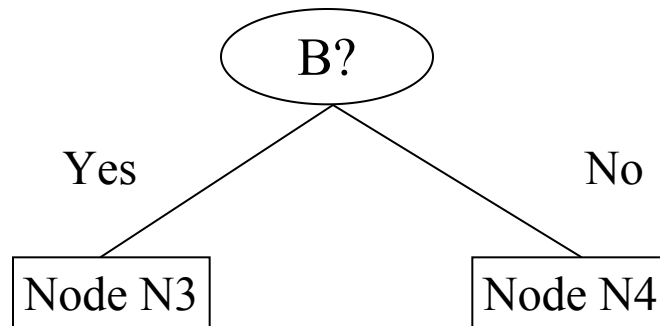
C0	N20
C1	N21

↓
M1

↓
M2



M12



C0	N30
C1	N31

C0	N40
C1	N41

↓
M3

↓
M4



M34

Gain = M0 – M12 vs M0 – M34

Measure of Impurity: Gini Index

- Gini Index for a given node t :

$$GINI(t) = 1 - \sum_j [p(j | t)]^2$$

(NOTE: $p(j | t)$ is the relative frequency of class j at node t).

- Maximum ($1 - 1/n_c$) when records are equally distributed among all classes, implying least interesting information
- Minimum (0) when all records belong to one class, implying most interesting information

C1	0
C2	6
Gini=0.000	

C1	1
C2	5
Gini=0.278	

C1	2
C2	4
Gini=0.444	

C1	3
C2	3
Gini=0.500	

Examples for computing GINI

$$GINI(t) = 1 - \sum_j [p(j | t)]^2$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Gini = 1 - P(C1)^2 - P(C2)^2 = 1 - 0 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Gini = 1 - (2/6)^2 - (4/6)^2 = 0.444$$

Splitting Based on GINI

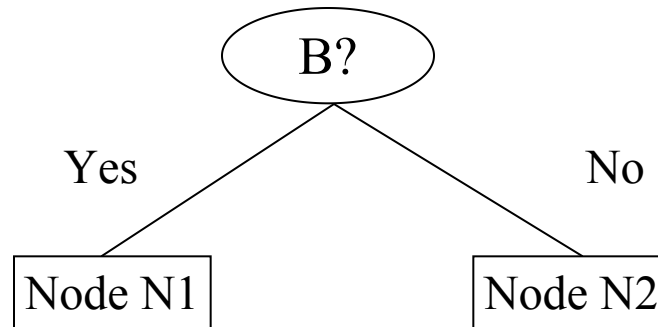
- Used in CART, SLIQ, SPRINT.
- When a node p is split into k partitions (children), the quality of split is computed as,

$$GINI_{split} = \sum_{i=1}^k \frac{n_i}{n} GINI(i)$$

where, n_i = number of records at child i ,
 n = number of records at node p .

Example: GINI Index Computation

- Splits into two partitions
- Effect of Weighing partitions:
 - Larger and Purer Partitions are sought for.



	Parent
C1	6
C2	6
Gini = 0.500	

$$\begin{aligned}
 \text{Gini}(N1) &= 1 - (5/6)^2 - (2/6)^2 \\
 &= 0.194
 \end{aligned}$$

$$\begin{aligned}
 \text{Gini}(N2) &= 1 - (1/6)^2 - (4/6)^2 \\
 &= 0.528
 \end{aligned}$$

	N1	N2
C1	5	1
C2	2	4
Gini=0.333		

$$\begin{aligned}
 \text{Gini(Children)} &= 7/12 * 0.194 + \\
 &\quad 5/12 * 0.528 \\
 &= 0.333
 \end{aligned}$$

Categorical Attributes: Computing Gini Index

- For each distinct value, gather counts for each class in the dataset
- Use the count matrix to make decisions

Multi-way split

	CarType		
	Family	Sports	Luxury
C1	1	2	1
C2	4	1	1
Gini	0.393		

Two-way split
(find best partition of values)

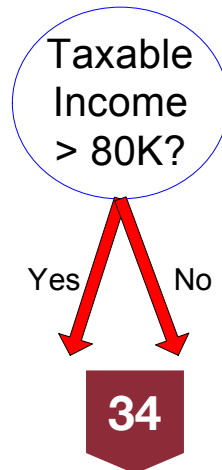
	CarType	
	{Sports, Luxury}	{Family}
C1	3	1
C2	2	4
Gini	0.400	

	CarType	
	{Sports}	{Family, Luxury}
C1	2	2
C2	1	5
Gini	0.419	

Continuous Attributes: Computing Gini Index

- Use Binary Decisions based on one value
- Several Choices for the splitting value
 - Number of possible splitting values = Number of distinct values
- Each splitting value has a count matrix associated with it
 - Class counts in each of the partitions, $A < v$ and $A \geq v$
- Simple method to choose best v
 - For each v , scan the database to gather count matrix and compute its Gini index
 - Computationally Inefficient! Repetition of work.

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



Continuous Attributes: Computing Gini Index...

- For efficient computation: for each attribute,
 - Sort the attribute on values
 - Linearly scan these values, each time updating the count matrix and computing gini index
 - Choose the split position that has the least gini index

		Cheat	No		No		No		Yes		Yes		Yes		No		No		No		No			
			Taxable Income																					
Sorted Values Split Positions	→		60		70		75		85		90		95		100		120		125		220			
	→		55		65		72		80		87		92		97		110		122		172		230	
			<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>	<=	>
		Yes	0	3	0	3	0	3	0	3	1	2	2	1	3	0	3	0	3	0	3	0	3	0
		No	0	7	1	6	2	5	3	4	3	4	3	4	3	4	4	3	5	2	6	1	7	0
		Gini	0.420		0.400		0.375		0.343		0.417		0.400		<u>0.300</u>		0.343		0.375		0.400		0.420	

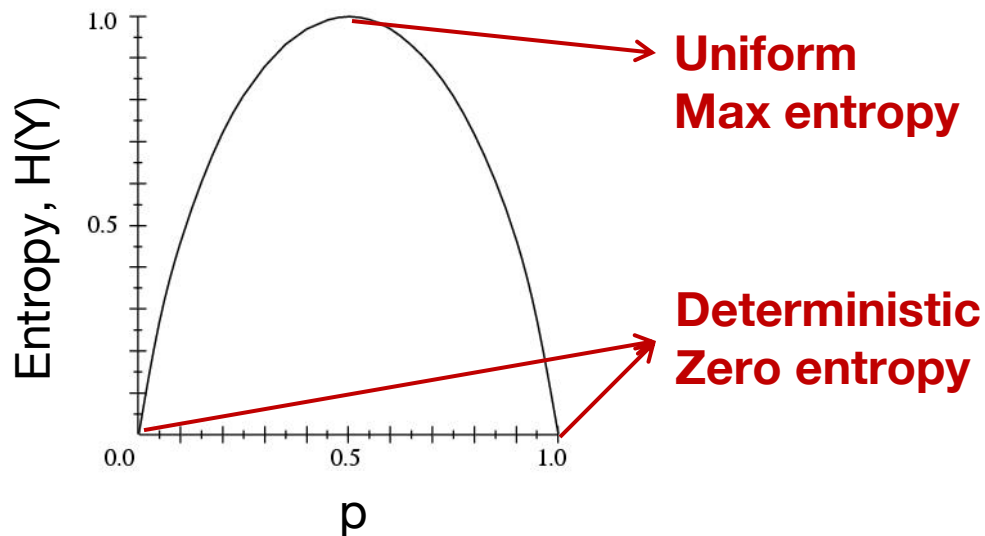
Entropy

- Entropy of a random variable Y

$$H(Y) = - \sum_y P(Y = y) \log_2 P(Y = y)$$

***More uncertainty,
more entropy!***

$Y \sim \text{Bernoulli}(p)$



$$H(Y) = -p \log_2 p - (1 - p) \log_2 (1 - p)$$

Splitting Criteria based on Entropy

- Entropy at a given node t :

$$Entropy(t) = -\sum_j p(j | t) \log p(j | t)$$

(NOTE: $p(j | t)$ is the relative frequency of class j at node t).

- Measures homogeneity of a node.
 - Maximum ($\log n_c$) when records are equally distributed among all classes implying least information
 - Minimum (0.0) when all records belong to one class, implying most information
- Entropy based computations are similar to the GINI index computations

Examples for computing Entropy

$$Entropy(t) = -\sum_j p(j | t) \log_2 p(j | t)$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Entropy = -0 \log 0 - 1 \log 1 = -0 - 0 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Entropy = - (1/6) \log_2 (1/6) - (5/6) \log_2 (1/6) = 0.65$$

C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Entropy = - (2/6) \log_2 (2/6) - (4/6) \log_2 (4/6) = 0.92$$

Splitting Based on Entropy

- Information Gain (信息增益):

$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^k \frac{n_i}{n} Entropy(i) \right)$$

Parent Node, p is split into k partitions;
 n_i is number of records in partition i

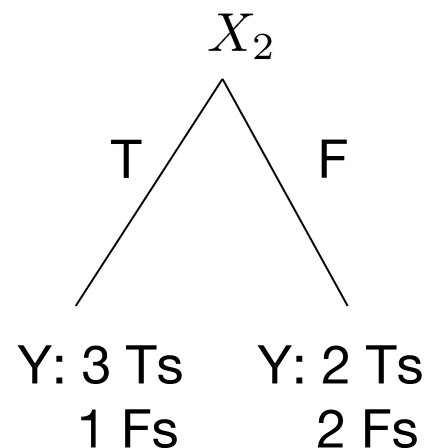
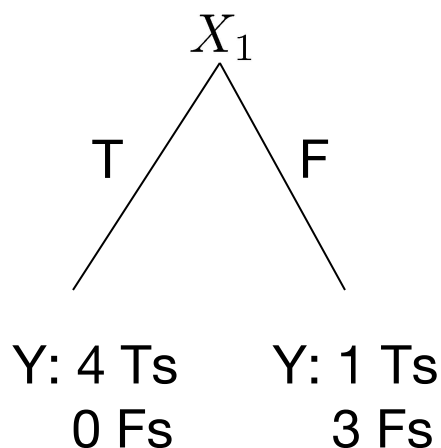
- Measures Reduction in Entropy achieved because of the split
- Choose the split that achieves most reduction (maximizes GAIN)
- Feature which yields maximum reduction in entropy provides maximum information about classification label

Example: Information Gain

$$H(Y) = -\frac{5}{8} \log_2 \frac{5}{8} - \frac{3}{8} \log_2 \frac{3}{8}$$

$$H(Y | X_i) = - \sum_x P(X_i = x) \sum_y P(Y = y | X_i = x) \log_2 P(Y = y | X_i = x)$$

X_1	X_2	Y
T	T	T
T	F	T
T	T	T
T	F	T
F	T	T
F	F	F
F	T	F
F	F	F



$$H(Y|X_1) = -\frac{1}{2}[1 \log_2 1 + 0 \log_2 0] - \frac{1}{2}[\frac{1}{4} \log_2 \frac{1}{4} + \frac{3}{4} \log_2 \frac{3}{4}]$$

$$H(Y|X_2) = -\frac{1}{2}[\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4}] - \frac{1}{2}[\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2}]$$

$$H(Y|X_1) < H(Y|X_2)$$

> 0

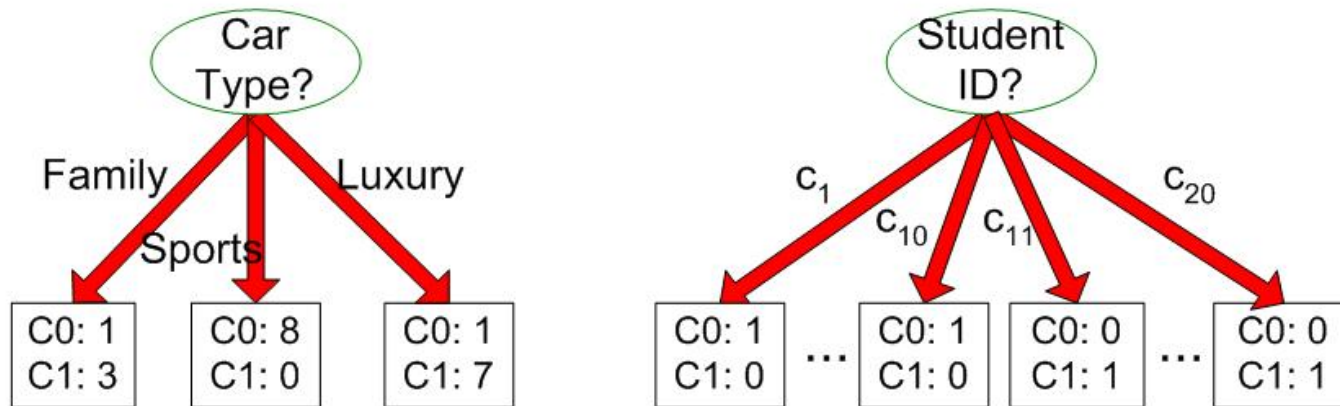


Information Gain

- Used in ID3
- Disadvantage: Tends to prefer splits that result in large number of partitions, each being small but pure.

Example

Before Splitting: 10 records of class 0,
10 records of class 1



Which test condition is the best based on information gain?

Splitting Based on INFO...

- Information Gain Ratio(信息增益率):

$$GainRATIO_{split} = \frac{GAIN_{Split}}{SplitINFO} \quad SplitINFO = -\sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n}$$

Parent Node, p is split into k partitions

n_i is the number of records in partition i

- Adjusts Information Gain by the entropy of the partitioning (SplitINFO). Higher entropy partitioning (large number of small partitions) is penalized!
- **Used in C4.5**
- Designed to overcome the disadvantage of Information Gain

Splitting Criteria based on Classification Error

- Classification error at a node t :

$$Error(t) = 1 - \max_i P(i | t)$$

- Measures misclassification error made by a node.
 - Maximum ($1 - 1/n_c$) when records are equally distributed among all classes, implying least interesting information
 - Minimum (0.0) when all records belong to one class, implying most interesting information

Examples for Computing Error

$$Error(t) = 1 - \max_i P(i | t)$$

C1	0
C2	6

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Error = 1 - \max(0, 1) = 1 - 1 = 0$$

C1	1
C2	5

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Error = 1 - \max(1/6, 5/6) = 1 - 5/6 = 1/6$$

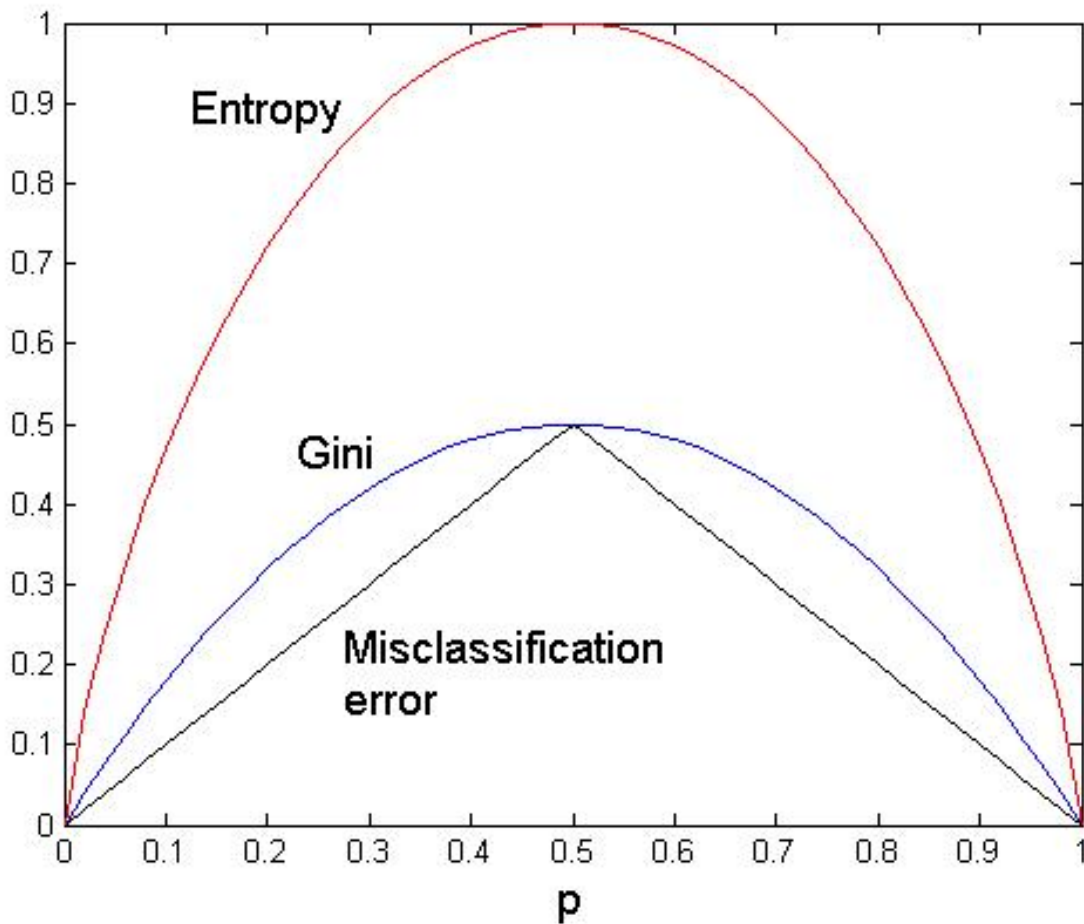
C1	2
C2	4

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Error = 1 - \max(2/6, 4/6) = 1 - 4/6 = 1/3$$

Comparison among Splitting Criteria

For a 2-class problem:



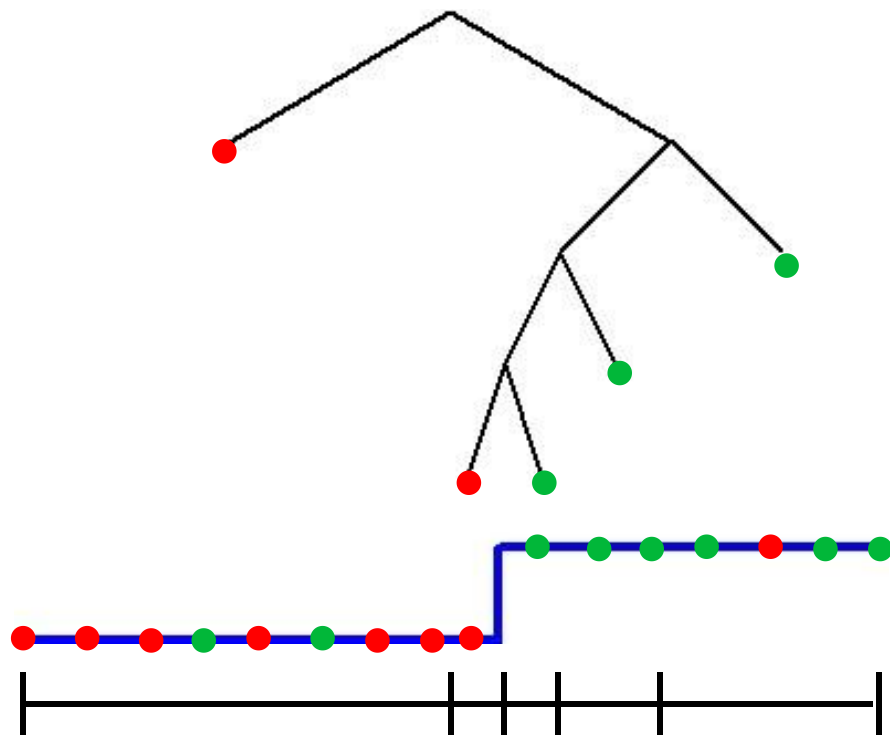
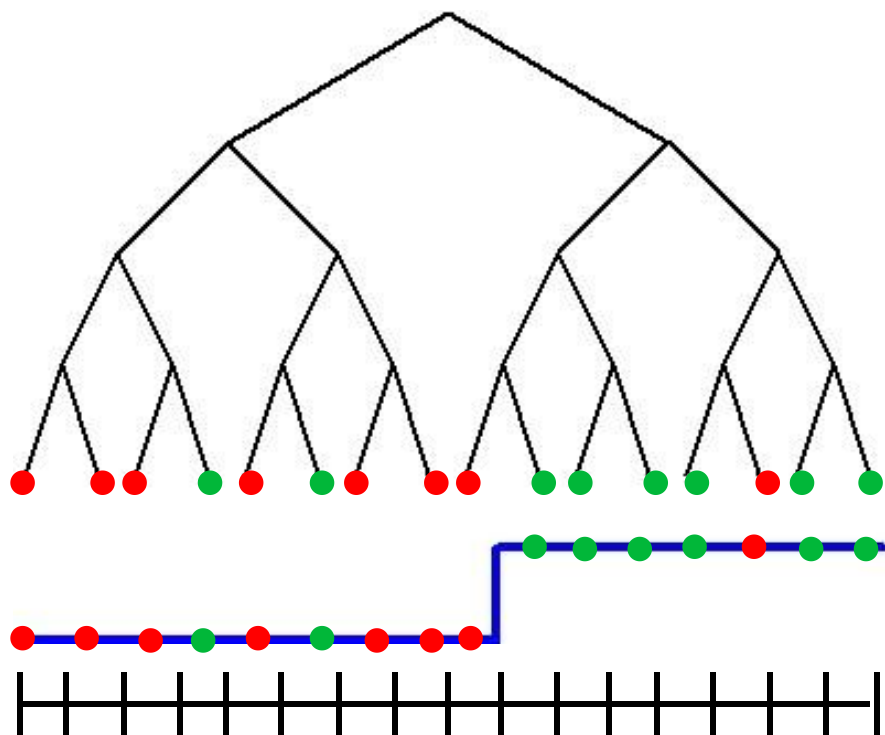


Tree Induction

- Greedy strategy.
 - Split the records based on an attribute test that optimizes certain criterion.
- Issues
 - Determine how to split the records
 - How to specify the attribute test condition?
 - How to determine the best split?
 - Determine when to stop splitting

Decision Trees - Overfitting

One training example per leaf – overfits, need compact/pruned decision tree



How to Address Overfitting

- Pre-Pruning (Early Stopping Rule)

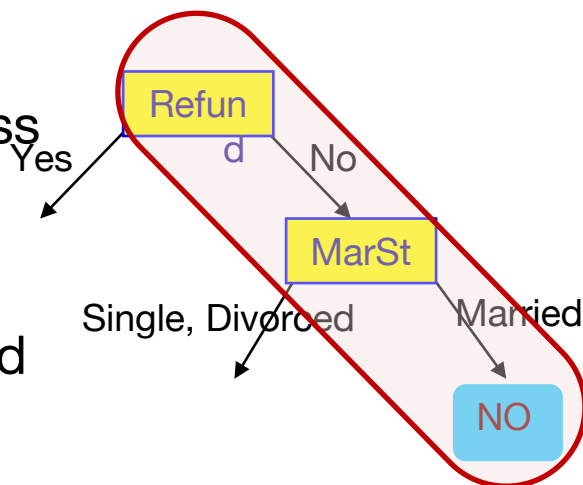
- Stop the algorithm before it becomes a fully-grown tree

- Typical stopping conditions for a node:

- Stop if all instances belong to the same class
- Stop if all the attribute values are the same
- Stop when fixed depth is satisfied
- Stop when fixed number of leaves is satisfied

- More restrictive conditions:

- Stop if number of instances is less than some user-specified threshold
- Stop if expanding the current node does not improve impurity measures (e.g., Gini or information gain).





How to Address Overfitting

- **Post-pruning**
 - Grow decision tree to its entirety
 - Trim the nodes of the decision tree in a bottom-up fashion
 - If **generalization error** improves after trimming, replace sub-tree by a leaf node.
 - Class label of leaf node is determined from majority class of instances in the sub-tree

Post-pruning

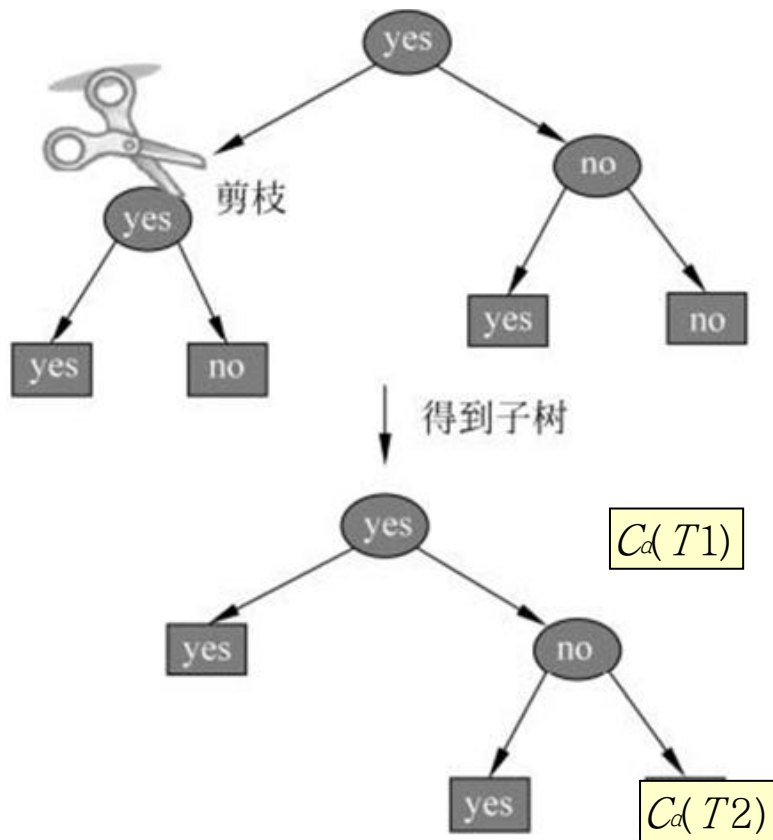
- Penalize complex models by introducing cost

$$\hat{f} = \arg \min_T \left\{ \underbrace{\frac{1}{n} \sum_{i=1}^n \text{loss}(\hat{f}_T(X_i), Y_i)}_{\text{log likelihood}} + \underbrace{\text{pen}(T)}_{\text{cost}} \right\}$$

$$\begin{aligned} \text{loss}(\hat{f}_T(X_i), Y_i) &= (\hat{f}_T(X_i) - Y_i)^2 && \text{regression} \\ &= \mathbf{1}_{\hat{f}_T(X_i) \neq Y_i} && \text{classification} \end{aligned}$$

$\text{pen}(T) = \text{cp} \cdot |T|$ penalize trees with more nodes;
cp : complexity penalty parameter
(tuned by cross-validation)

Post-pruning



$$C_\alpha(T) = \sum_{t=1}^{|T|} N H(T) + \alpha |T|$$

$$H(T) = - \sum_{k=1}^K \frac{N_{tk}}{N} \log \frac{N_{tk}}{N}$$

So far...

- What does a classification tree represent
- Given a tree, how do we assign label to a test point
- How do we learn a tree from training data
- What will happen if we want to use classification tree to deal with continuous target label?

=> CART !!!!

- Decision Tree
 - ID3, C4.5, CART
 - Entropy/Gini Index/Maximum Error
 - Pruning and overfitting
- Regression Tree
- Case Study



Regression Tree

- **Regression Tree**

- **Tree-based modeling for continuous target variable**
- **Find split that produces greatest separation in**

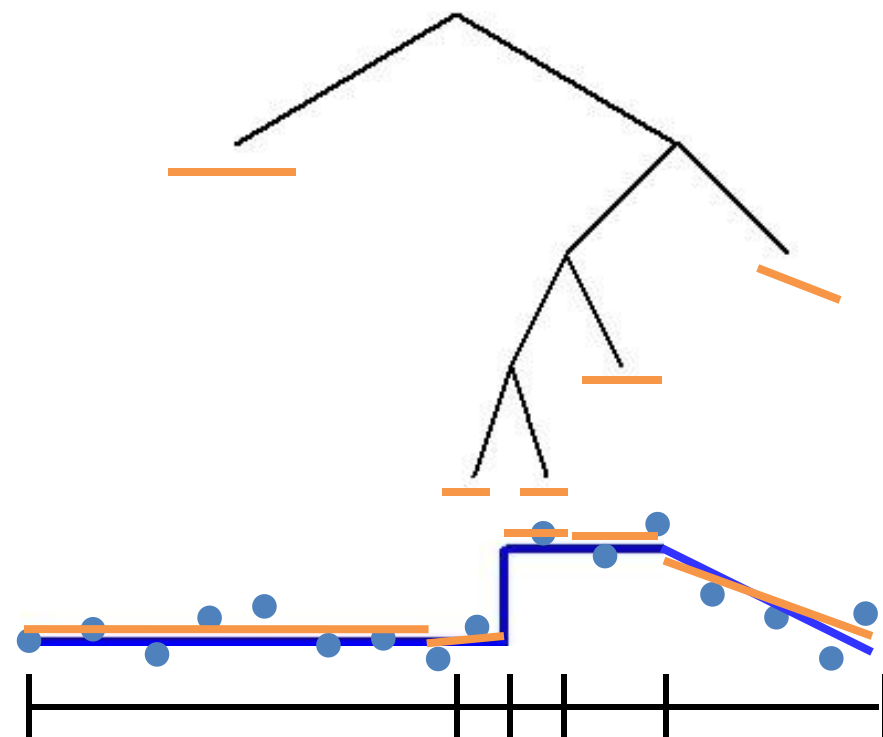
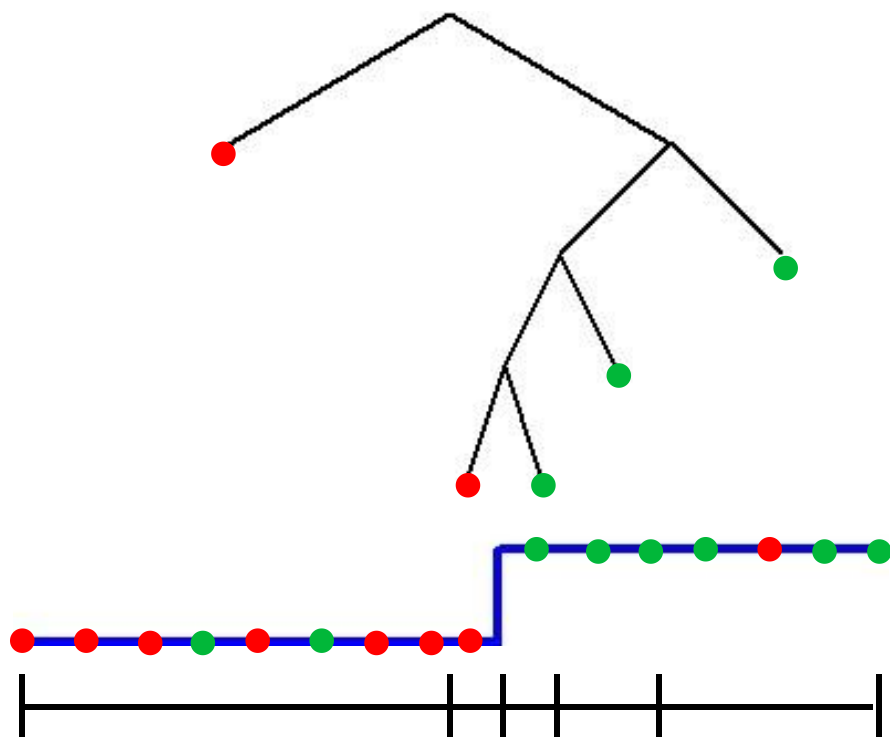
$$\sum [y - E(y)]^2$$

- **Every record in a node can be:**
 - (1) assigned the same value (average)
 - (2) assigned the predicted value of a linear regression

How to assign label to each leaf

Classification – Majority vote

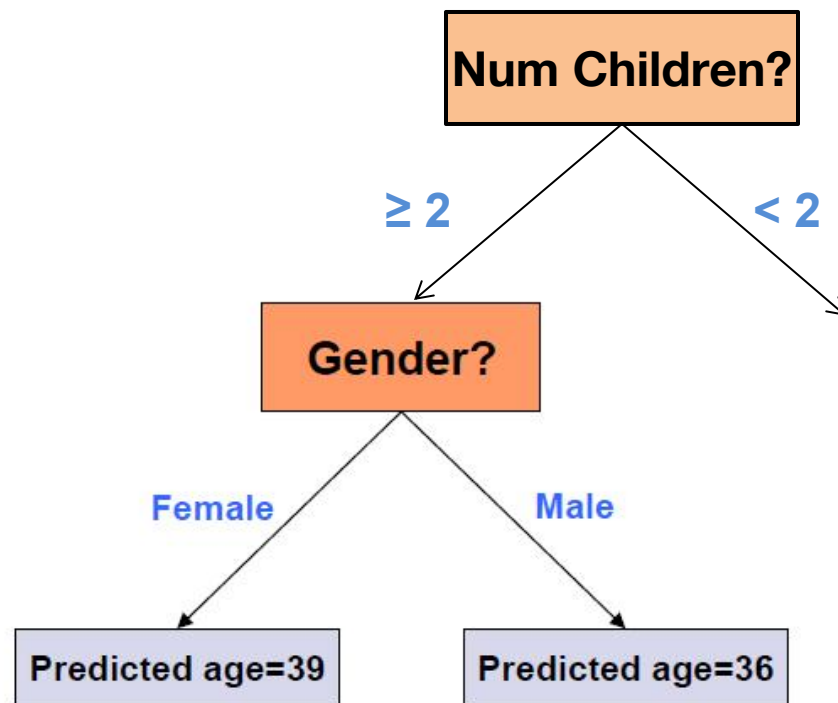
Regression – Constant/ Linear



Regression trees

Y

Gender	Rich?	Num. Children	# travel per yr.	Age
F	No	2	5	38
M	No	0	2	25
M	Yes	1	0	72
:	:	:	:	:



Average (fit a constant) using training data at the leaves



Classification and Regression Tree (CART)

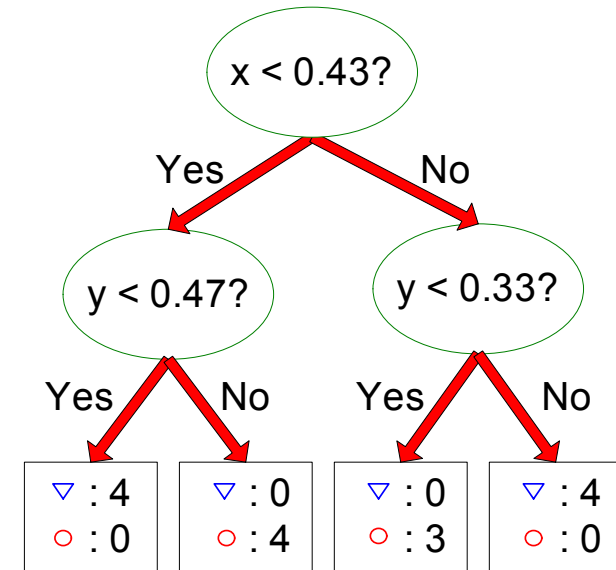
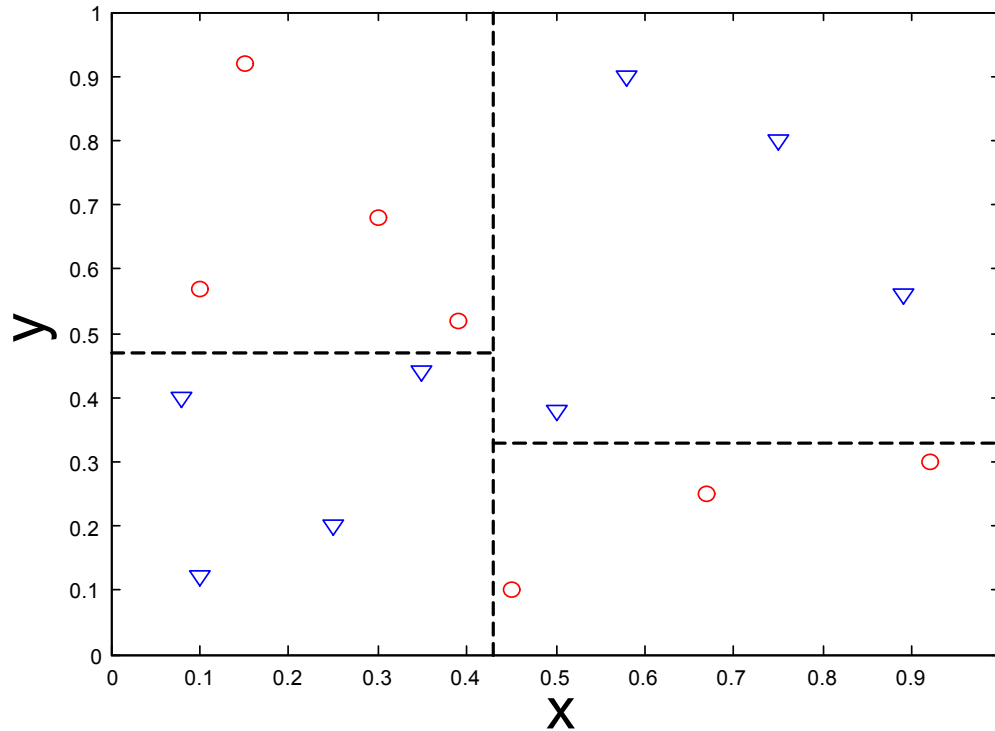
- **CART:** Combine classification tree and regression tree
- Using binary split
- Gini index is always used when dealing with discrete class label



Decision Tree Based Classification

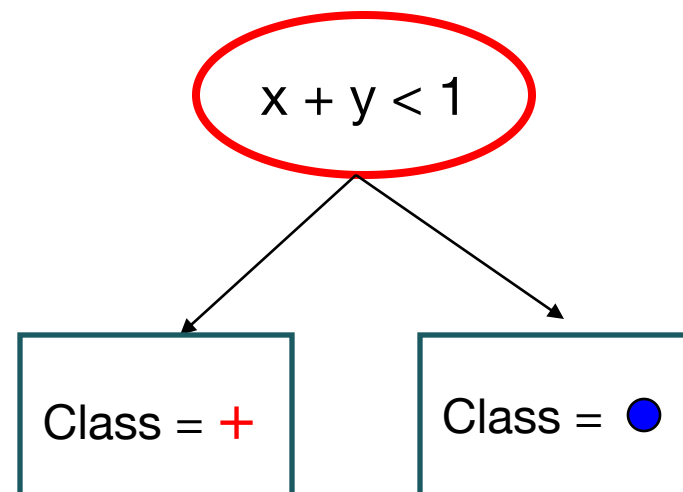
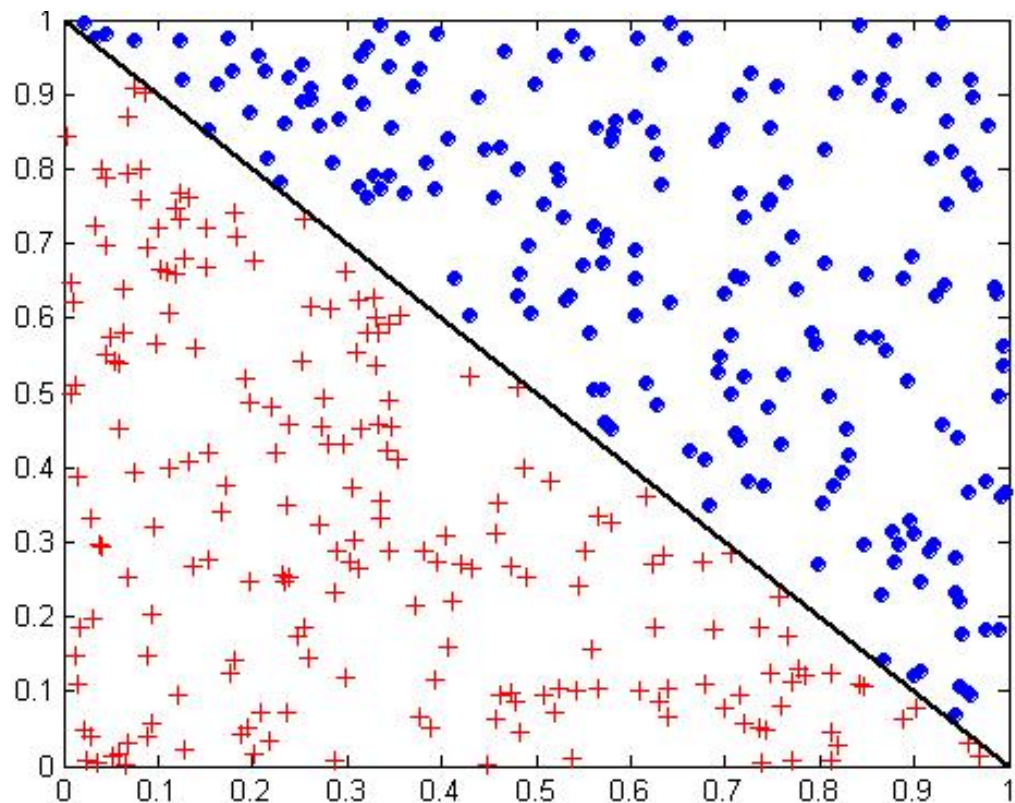
- Advantages:
 - Inexpensive to construct
 - Extremely fast at classifying unknown records
 - Easy to interpret for small-sized trees
 - Accuracy is comparable to other classification techniques for many simple data sets

Decision Boundary



- Border line between two neighboring regions of different classes is known as decision boundary
- Decision boundary is parallel to axes because test condition involves a single attribute at-a-time

Oblique Decision Trees



- Test condition may involve multiple attributes
- More expressive representation
- Finding optimal test condition is computationally expensive

- Decision Tree
 - ID3, C4.5, CART
 - Entropy/Gini Index/Maximum Error
 - Pruning and overfitting
- Regression Tree
- Case Study

Case Study: Spam Email Detection

1. **spam**: Indicator for whether the email was spam.
2. **to_multiple**: Indicator for whether the email was addressed to more than one recipient.
3. **from**: Whether the message was listed as from anyone (this is usually set by default for regular outgoing email).
4. **cc**: Indicator for whether anyone was CCed.
5. **sent_email**: Indicator for whether the sender had been sent an email in the last 30 days.
6. **image**: Indicates whether any images were attached.
7. **attach**: Indicates whether any files were attached.
8. **dollar**: Indicates whether a dollar sign or the word 'dollar' appeared in the email.
9. **winner**: Indicates whether "winner" appeared in the email.
10. **inherit**: Indicates whether "inherit" (or an extension, such as inheritance) appeared in the email.
11. **password**: Indicates whether "password" appeared in the email.
12. **num_char**: The number of characters in the email, in thousands.
13. **line_breaks**: The number of line breaks in the email (does not count text wrapping).
14. **format**: Indicates whether the email was written using HTML (e.g. may have included bolding or active links) or plaintext.
15. **re_subj**: Indicates whether the subject started with "Re:", "RE:", "re:", or "rE":
16. **exclaim_subj**: Indicates whether there was an exclamation point in the subject.
17. **urgent_subj**: Indicates whether the word "urgent" was in the email subject.
18. **exclaim_mess**: The number of exclamation points in the email message.
19. **number**: Factor variable saying whether there was no number, a small number (under 1 million), or a big number.

Case Study: Tree-Based Models

```
In [14]: from sklearn.tree import DecisionTreeClassifier
         # Fitting a decision tree model with default parameters
         dt = DecisionTreeClassifier()
         dt.fit(X_train, y_train)
```

```
Out[14]: DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
                                max_features=None, max_leaf_nodes=None,
                                min_impurity_decrease=0.0, min_impurity_split=None,
                                min_samples_leaf=1, min_samples_split=2,
                                min_weight_fraction_leaf=0.0, presort=False, random_state=None,
                                splitter='best')
```

```
In [26]: # import the regressor
         from sklearn.tree import DecisionTreeRegressor

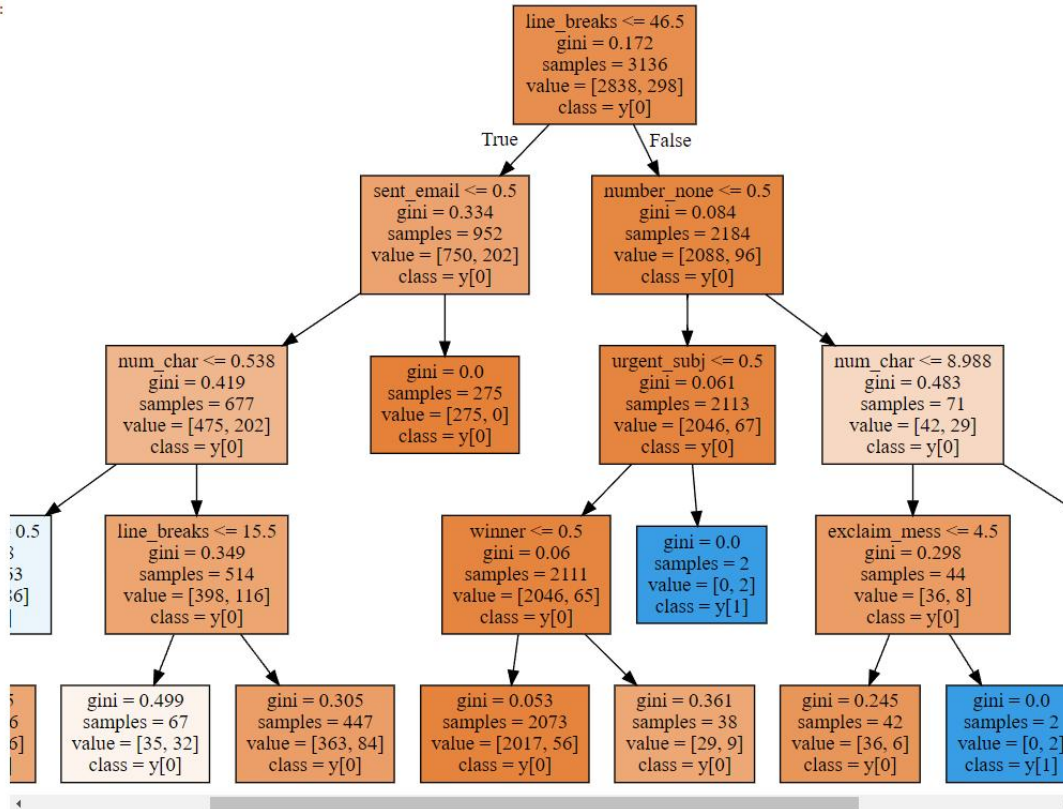
         # create a regressor object
         dt = DecisionTreeRegressor(random_state = 0)
         # fit the regressor with X and Y data
         dt.fit(X_train, y_train)
```

```
Out[26]: DecisionTreeRegressor(criterion='mse', max_depth=None, max_features=None,
                                max_leaf_nodes=None, min_impurity_decrease=0.0,
                                min_impurity_split=None, min_samples_leaf=1,
                                min_samples_split=2, min_weight_fraction_leaf=0.0,
                                presort=False, random_state=0, splitter='best')
```


Case Study: Email Data

```
In [19]: from sklearn import tree
import graphviz
dot_data = tree.export_graphviz(dt, feature_names = X.columns, filled = True, class_names = True, out_file=None)
graph = graphviz.Source(dot_data)
graph
```

Out[19]:

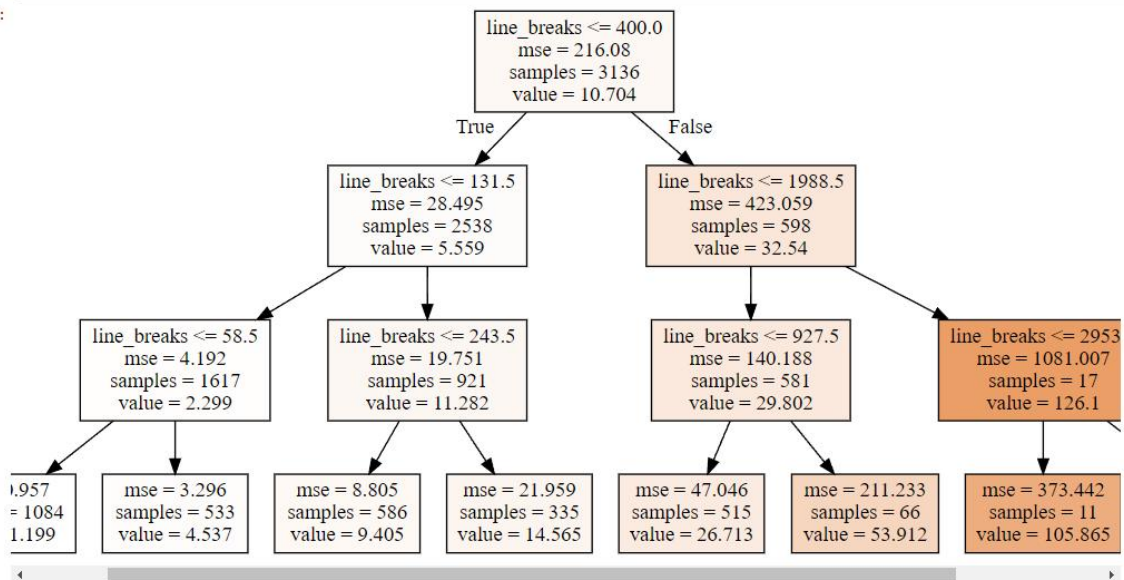


Case Study: Email Data

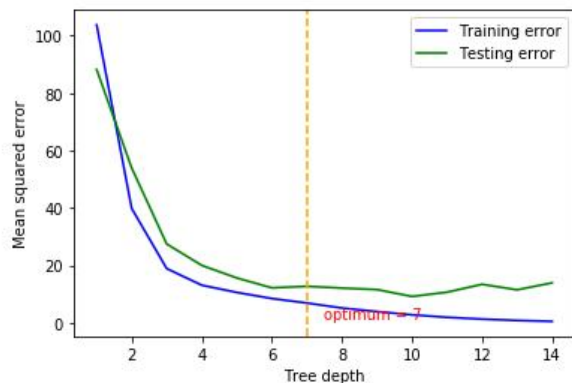
```
In [28]: # Fitting a decision tree model with max_depth = 3
dt = DecisionTreeRegressor(random_state = 0, max_depth=3)
# fit the regressor with X and Y data
dt.fit(X_train, y_train)

dot_data = tree.export_graphviz(dt, feature_names = X.columns, filled = True, class_names = True, out_file=None)
graph = graphviz.Source(dot_data)
graph
```

Out[28]:



Hyperparameter Tuning



The End

■ THANK YOU

Any questions?

